Study book

Topic 5: Debugging fundamentals (Debug Lab A)

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 1, topic 5 of 20.

Brief overview

What this lesson is about. Finding bugs **without** `Console.WriteLine`: a repeatable method (reproduce, locate, hypothesize, verify, fix, re-run) and the Rider toolbox (Step Into/Over/Out, Smart Step Into, Run to Cursor, Watches, Evaluate Expression, conditional breakpoints, exception breakpoints, Frames, logpoints).
What you did. Debug Lab A: a `DashboardReport` with **three planted bugs** and a runner that reproduces three bug reports on realistic data (300 imported tasks). You found all three in Rider; the fixes were then applied and verified.
The three bugs, each tied to an earlier topic: **integer division** (C# types), **an empty string from a double space** in imported data (defensive reading), **a copied reference** that let a read-only report delete data (topics 1, 2, 3).
Bonus gotchas found on the way: `Math.Round` uses banker's rounding; `dotnet run file.cs` can run a **stale cached build**.

Contents

Part	What is inside
1	Why a debugger instead of Console.WriteLine
2	The debugging method
3	The Rider toolbox (with the breakpoint popup from your Rider)
4	Lab setup: files, data, how to run, how to debug
5	Bug 1: integer division
6	Bug 2: the double space in task 158
7	Bug 3: the read-only report that deleted data
8	Two bonus gotchas: banker's rounding and stale builds
9	Your answers, corrections, interview Q&A
10	Summary, checklist, what comes next

Part 1. Why a debugger instead of Console.WriteLine

Console.WriteLine / console.log	Debugger
guess **in advance** what to print	see **every** variable at the paused moment
change code, rebuild, rerun for every new question	ask new questions **while paused** (Watches, Evaluate Expression)
shows values, not **how you got here**	the **call stack** shows the chain of calls
prints every loop iteration (noise)	**conditional breakpoints** stop only on the interesting one
leftover prints get committed	nothing to clean up

Logs are for production, where you cannot attach a debugger (topic 17). Locally you use the debugger. Your answer: 'we don't need to know the place in advance, we have lots of tools; logs are for prod' (right).

Part 2. The method

1. **Reproduce** the bug reliably (the lab runner does this).
2. **Locate** where the wrong value first appears: start at the symptom, walk backwards (call stack, Step Into) until one line is right and the next is wrong.
3. **Hypothesize** in one sentence why it is wrong.
4. **Verify the hypothesis in the debugger before changing code** (Evaluate Expression with the corrected formula).
5. **Fix**, then **re-run the reproduction**; later a **unit test** keeps the bug from coming back (topic 13).

Common mistake

Your answer listed practical steps (run, exception breakpoint, precise breakpoints, step into) — that is the **locate** phase only. Missing: hypothesize, **verify before changing code**, fix, re-run. Verifying first is what separates debugging from guessing; you did it in bug 1.

Part 3. The Rider toolbox

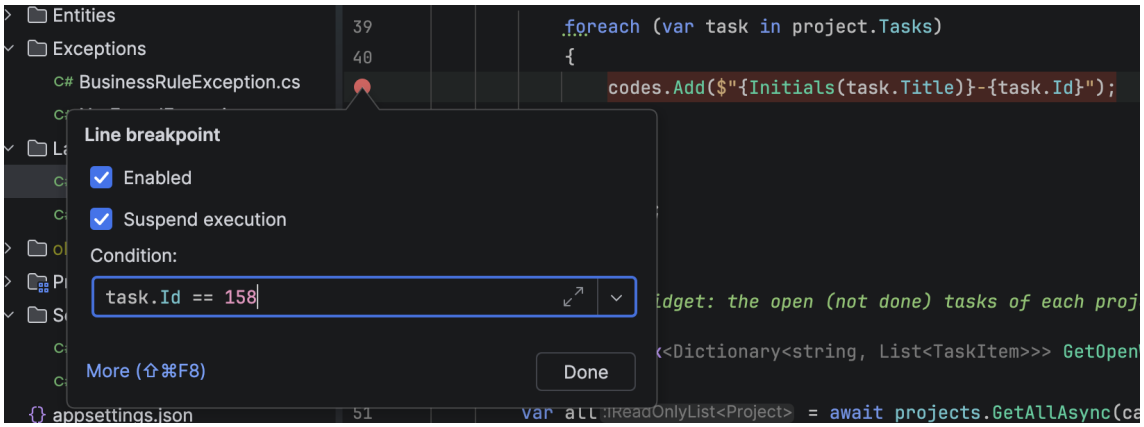
Tool	What it does	Use when
Breakpoint	pauses **before** a line runs	you know roughly where to look
Step Over	runs the line, stops at the next one in the same method	the call's internals don't matter
Step Into	enters the method called on the line	the bug may be inside that call
Smart Step Into	choose **which** call to enter on a line with several	`result.Add(new X(..., Calc(a, b)))`
Step Out	runs to the end of the method, stops in the caller	seen enough here
Run to Cursor	runs to the cursor's line: a **one-time breakpoint**	skip ahead (e.g. past a 300-item loop)

Tool	What it does	Use when
Watches	**pinned** expressions, re-evaluated **automatically** at every pause	track a value across steps (`project.Tasks.Count`)
Evaluate Expression	runs any C# expression now, in the paused context	test a hypothesis or a fix
Conditional breakpoint	pauses only when a condition is true	1 bad item in a big loop
Hit count	pauses only on the Nth hit	'fails on the 3rd call'
Logpoint / tracepoint	prints without pausing and without editing code	a log line without touching code
Exception breakpoint	pauses at the **throw**, locals alive, **even if caught later**	'it fails somewhere'
Frames (call stack)	click a caller frame to see **its** variables	'who called this with this value?'
Parallel Stacks	all threads and where they are	async / concurrency (topic 4)

Your Rider's menu for exception breakpoints: **Run > Stop on Exception...** (other versions: Run > View Breakpoints > + > .NET Exception Breakpoint). If a menu name differs: **Help > Find Action** (Cmd+Shift+A) and type the feature name.

Rider: the breakpoint popup

Right-click the red breakpoint dot to open the **Line breakpoint** popup (your Rider, Topic 5):
Enabled: switch the breakpoint off without deleting it.
Suspend execution: checked = pause here (normal breakpoint). Unchecked = do not pause; together with a log message in **More** it becomes a **logpoint / tracepoint** that prints without stopping and without editing code.
Condition: a C# expression evaluated at this line; the debugger pauses only when it is `true`, e.g. `task.Id == 158`. The arrow on the right shows conditions you used before; the expand icon gives a bigger editor.
More (Shift+Cmd+F8): the full breakpoint dialog (hit count, log message, all breakpoints in one list).
Done saves it. A conditional breakpoint gets a small mark on its red dot.



Your Rider: a conditional breakpoint on the codes.Add line, condition task.Id == 158 (Debug Lab A, bug 2).

Part 4. Lab setup

File	Role
src/ProjectHub.Api/Labs/DashboardReport.cs	report service for a future dashboard: Progress, Task codes, Open work widgets. Had 3 planted bugs (now fixed, marked 'LAB FIX')
src/ProjectHub.Api/Labs/DebugLabA.cs	runner: builds lab data and reproduces the three bug reports (not buggy)
playground/05-debug-lab.cs	runs the lab without the debugger

- **Lab data:** the sample projects + project 4 'Legacy import' with **300 tasks** (ids 8-307, every 3rd Done) 'copied from the old tracker'. Task **158** has the title `"Migrate legacy reports"` with a **double space**.
- **See the symptoms:** `dotnet run --no-cache playground/05-debug-lab.cs`.
- **Debug:** temporarily add `await ProjectHub.Api.Labs.DebugLabA.RunAsync();` to `Program.cs` before `app.Run();`, start with **Debug**.
- **Rules:** no `Console.WriteLine`; don't edit the buggy file before the cause is found and verified.

The symptoms (before the fixes):

```
=== Bug report 1: 'The progress widget shows 0% for projects that are half done' ===
  Website redesign     2/4   done -> 0%
  Mobile app           0/3   done -> 0%
  Internal tools       0/0   done -> 0%
  Legacy import      100/300 done -> 0%

=== Bug report 2: 'The task codes widget fails to load' ===
  FAILED: IndexOutOfRangeException: Index was outside the bounds of the array.

=== Bug report 3: 'After the Open work widget loads, done tasks vanish from the project page' ===
  Project 1 task count before the widget: 4, after: 2
```

Part 5. Bug 1: 0% progress (integer division)

Buggy code (as handed out):

```
private static int CalculatePercent(int done, int total)
{
    if (total == 0)
    {
        return 0;
    }

    return done / total * 100;
}
```

How it was found

1. Breakpoint on `result.Add(...)` in `GetProgressAsync`; for Website redesign the inputs were correct: `done = 2`, `total = 4`.
2. **Smart Step Into** `CalculatePercent`; **Evaluate Expression** on parts of the formula: `done / total` gives `0`.
3. Your reaction: 'WTF, 50 isn't int? WTF this didn't work done / total * 100?!' — that moment **is** the finding.

Why

```
done / total * 100        // evaluated left to right
= (2 / 4) * 100           // int / int = INTEGER division: the fraction is cut off
= 0 * 100
= 0
```

50 **is** an int; the problem is the intermediate step. When **both operands of `/` are int**, C# does integer division and **truncates toward zero** (`2 / 4 = 0`, `7 / 2 = 3`, `-7 / 2 = -3`, `299 / 300 = 0`). JS has one number type (double), so `2 / 4` is `0.5`.

Evaluate	Result	Why
`2 / 4`	`0`	int / int
`2 / 4.0`	`0.5`	one side is a double
`2 * 100 / 4`	`50`	multiply first
`100 * 100 / 300`	`33`	10000 / 300 truncated
`(int)Math.Round(100 * 100.0 / 300)`	`33`	double math, then round

Fixed code:

```
private static int CalculatePercent(int done, int total)
{
    if (total == 0)
    {
        return 0;
    }

    // LAB FIX (bug 1): was `done / total * 100`. int / int is integer division (2 / 4 == 0).
    // 100.0 makes it double math; AwayFromZero rounds 12.5 up to 13 (the default would give 12).
    return (int)Math.Round(done * 100.0 / total, MidpointRounding.AwayFromZero);
}
```

`100.0` makes it double math; `MidpointRounding.AwayFromZero` makes 12.5% show as 13% (see Part 8). The alternative `done * 100 / total` stays int but truncates (66.67 -> 66).

Part 6. Bug 2: the crash in 1 of 300 tasks

Buggy code:

```
private static string Initials(string title)
{
    var words = title.Split(' ');
    var letters = new char[words.Length];

    for (var i = 0; i < words.Length; i++)
    {
        letters[i] = char.ToUpperInvariant(words[i][0]);
    }

    return new string(letters);
}
```

How it was found

1. Exception breakpoint for `System.IndexOutOfRangeException` (**Run > Stop on Exception...**). Rider stopped in `Initials`, on `words[i][0]`, although the runner **catches** the exception.
2. Variables: `title = "Migrate legacy reports"`, `words = ["Migrate", "", "legacy", "reports"]`, `i = 1`.
3. **Frames:** clicked the caller frame `GetTaskCodesAsync` and read `task.Id = 158`.
4. Conditional breakpoint on `codes.Add(...)` with `task.Id == 158` (your screenshot in Part 3): the debugger stopped **once** out of 300+ tasks; Step Into `Initials`.

Why

Your diagnosis: 'double space, so after Split one element is an empty string, which has no length' (right). `""[0]` has no character 0, so `IndexOutOfRangeException`.

Fixed code (the solution you asked for):

```
private static string Initials(string title)
{
    // LAB FIX (bug 2): was `title.Split(' ')`. A double space produced an empty word, and ""[0] threw
    // IndexOutOfRangeException. Imported data does not pass through the service's validation.
    var words = title.Split(' ', StringSplitOptions.RemoveEmptyEntries);
    return words.Length == 0
        ? "?"
        : string.Concat(words.Select(w => char.ToUpperInvariant(w[0])));
}
```

- `StringSplitOptions.RemoveEmptyEntries` drops the empty pieces (JS: `title.split(' ').filter(Boolean)`).
- `words.Length == 0 ? "?"` guards a title made only of spaces.
- `string.Concat(words.Select(w => char.ToUpperInvariant(w[0])))`: the LINQ version of the loop (topic 2): map each word to its first letter, join them.
- Result: `"Migrate legacy reports"` (#158) -> `MLR-158`.

The bigger lesson

`ProjectService.CreateAsync` validates titles, but task 158 never went through it: it came from an **import**. Data from imports, databases and other systems does not always pass your validation: read it **defensively**, and clean it at the import boundary.

Part 7. Bug 3: the read-only report that deleted data

Buggy code:

```
public async Task<Dictionary<string, List<TaskItem>>> GetOpenWorkAsync(CancellationToken cancellationToken = default)
{
    var all = await projects.GetAllAsync(cancellationToken);
    var result = new Dictionary<string, List<TaskItem>>();

    foreach (var project in all)
    {
        var open = project.Tasks;
        open.RemoveAll(t => t.Status == TaskItemStatus.Done);
        result[project.Name] = open.OrderByDescending(t => t.CreatedAt).ToList();
    }

    return result;
}
```

How to find it

1. Breakpoint on `var open = project.Tasks;`.
2. **Watches:** `project.Tasks.Count` and `ReferenceEquals(open, project.Tasks)`.
3. Step Over line by line: `ReferenceEquals` is **true**, and `project.Tasks.Count` flips from **4 to 2** exactly on the `RemoveAll` line.

Why

`var open = project.Tasks;` copies the **reference**, not the list (topic 1: reference types). `open` and `project.Tasks` are two names for **the same list**, so `RemoveAll` deleted tasks from the service's real data. Topic 3's 'entities leak' warning came true; topic 2's `Where` would have avoided it because it builds a **new** sequence.

Fixed code:

```
public async Task<Dictionary<string, List<TaskItem>>> GetOpenWorkAsync(CancellationToken cancellationToken = default)
{
    var all = await projects.GetAllAsync(cancellationToken);
    var result = new Dictionary<string, List<TaskItem>>();

    foreach (var project in all)
    {
        // LAB FIX (bug 3): was `var open = project.Tasks; open.RemoveAll(...)`. That copied the REFERENCE,
        // so RemoveAll deleted tasks from the service's real list. Where() builds a new sequence instead.
        result[project.Name] = project.Tasks
            .Where(t => t.Status != TaskItemStatus.Done)
            .OrderByDescending(t => t.CreatedAt)
            .ToList();
    }

    return result;
}
```

Common mistake

Your answer 'RemoveAll was called on the original project.Tasks' said **what** happened. In the interview say **why**: '`var open = project.Tasks` copied the reference, not the list, so `RemoveAll` on `open` removed tasks from the service's real list.' You watched the count 'with your eyes'; a **Watch** would have shown the change automatically.

Lab output after all three fixes:

```
=== Bug report 1: 'The progress widget shows 0% for projects that are half done' ===
  Website redesign     2/4   done -> 50%
  Mobile app           0/3   done -> 0%
  Internal tools       0/0   done -> 0%
  Legacy import      100/300 done -> 33%

=== Bug report 2: 'The task codes widget fails to load' ===
  307 codes, first: DM-1, task 158: MLR-158

=== Bug report 3: 'After the Open work widget loads, done tasks vanish from the project page' ===
  Project 1 task count before the widget: 4, after: 4
```

Part 8. Two bonus gotchas found while fixing

8.1 Math.Round is banker's rounding

```
Math.Round(12.5)                                = 12   <- !
Math.Round(13.5)                                = 14
Math.Round(12.5, MidpointRounding.AwayFromZero)  = 13
```

By default .NET rounds an exact `.5` to the nearest **even** number (`MidpointRounding.ToEven`, 'banker's rounding'). JS `Math.round(12.5)` gives 13. Use `MidpointRounding.AwayFromZero` for 'school rounding'. It matters for percentages and money.

8.2 A stale build ran old code

After the fixes, `dotnet run playground/05-debug-lab.cs` still printed **all three bugs**, although the fixes were in the file. The file-based app reused a **cached build**: the playground file itself had not changed, so it did not rebuild the `src/` code it references. `dotnet run --no-cache ...` rebuilt and showed the fixed output.

- All playground headers, lessons and CLAUDE.md now use `dotnet run --no-cache playground/...`.
- **Debugging lesson:** when behaviour does not match the code you are reading, **suspect a stale build first** (frontend equivalent: stale cache, dev server not reloaded).

Part 9. Your answers, corrections, interview Q&A

9.1 CHECK round 1 and re-check

Question	Your answer	Complete answer
1. Debugger vs Console.WriteLine	no need to know the place in advance; many tools; logs are for prod (right)	+ ask new questions while paused; see how you got here (call stack)
2. The method	run, exception breakpoint, precise breakpoints, step into (locate only)	reproduce, locate, hypothesize, **verify**, fix, re-run
3. Over / Into / Out / Run to Cursor	first three right; Run to Cursor 'doesn't matter' (**wrong**); re-check: one-time breakpoint to skip ahead (right)	lab example: cursor on `return codes;` skips the loop
4. Conditional breakpoint	triggers only when the condition is true; e.g. id 158 in a 300-item loop (right)	
5. Exception breakpoint vs console	'see all variables' / re-check: stops at the throw with locals alive (half)	+ stops **even if the exception is caught later** (the runner caught bug 2)
6. Watch vs Evaluate	Evaluate right; Watch 'didn't use, watched with eyes'	Watch = **pinned** expression re-evaluated **automatically** at every step
7. Frames	'not interesting'	you **did** it in bug 2: click the caller frame to see its variables (task.Id 158)
8. Concepts	int/int 'rounded down'; double space; mutated original data	integer division **truncates toward zero**; empty string from Split; **copied reference**

9.2 Interview questions with model answers

Q: How do you debug a bug locally?

Reproduce it, locate where the value first goes wrong (breakpoints, call stack, Step Into), form a hypothesis, verify it in the debugger (Evaluate), fix, re-run the reproduction (and add a test).

Q: Why not Console.WriteLine?

You must guess in advance and rebuild for every question; the debugger shows all state, the call stack, and lets you ask new questions while paused. Logs are for production.

Q: Step Over vs Step Into vs Step Out vs Run to Cursor?

Over: next line in this method. Into: enter the called method. Out: finish this method, back to the caller. Run to Cursor: one-time breakpoint at the cursor to skip ahead.

Q: What is a conditional breakpoint?

A breakpoint that pauses only when a condition is true, e.g. task.Id == 158 inside a loop of 300 items.

Q: What does an exception breakpoint give you?

It stops at the throw with all locals alive, even if the exception is caught later; a console stack trace has no variable values.

Q: Watch vs Evaluate Expression?

Watch: pinned expression re-evaluated at every step. Evaluate: a one-off expression run now, to test a hypothesis or a fix.

Q: How do you find out who called a method with a bad value?

Frames / call stack: click the caller frame to see its variables.

Q: What is a logpoint (tracepoint)?

A breakpoint that logs a message without pausing and without editing code (uncheck Suspend execution, add a log message).

Q: What is 2 / 4 in C#? Why?

0: int / int is integer division, truncating toward zero. Use a double operand (2 / 4.0) or multiply first.

Q: What does Math.Round(2.5) return?

2: banker's rounding (to even) by default; use MidpointRounding.AwayFromZero for 3.

Q: How can Split(' ') cause a crash?

Consecutive spaces produce empty strings; indexing ""[0] throws. Use StringSplitOptions.RemoveEmptyEntries.

Q: Why did a read-only report delete data?

`var open = project.Tasks` copied the reference to the same list; RemoveAll changed the original. Use Where (new sequence) or copy the list.

Q: Why validate or read data defensively even if the service validates input?

Data can enter by other paths (imports, database, other systems) that bypass the service's validation.

Q: The code looks fixed but the old behaviour remains. First suspect?

A stale build or cache: force a rebuild (e.g. dotnet run --no-cache).

Part 10. Summary

Key takeaways

Method: reproduce, locate, hypothesize, **verify in the debugger**, fix, re-run.
Tools: Step Over/Into/Out, Smart Step Into, Run to Cursor, **Watches** (pinned, automatic), **Evaluate Expression** (test a fix while paused), **conditional breakpoints**, hit count, logpoints, **exception breakpoints** (stop at the throw, even if caught), **Frames** (caller's variables).
Bug 1: int / int truncates: `(2 / 4) * 100 = 0`. Fix with double math + `MidpointRounding.AwayFromZero`.
Bug 2: `Split(' ')` on a double space gives `""`; `""[0]` throws. Fix with `RemoveEmptyEntries`. Imported data skips your validation.
Bug 3: `var open = project.Tasks` copies a reference; `RemoveAll` mutated the real data. Fix with `Where`.
Gotchas: `Math.Round` is banker's rounding; a stale build can run old code (`--no-cache`).

Checklist: can you do this without notes?

- Explain the debugging method in 5-6 steps, including 'verify before changing code'.
- Set a conditional breakpoint and an exception breakpoint in Rider.
- Use Frames to find which caller passed a bad value.
- Use a Watch to see on which line a value changes.
- Explain each lab bug in one sentence (concept + fix).
- Explain banker's rounding and when to use AwayFromZero.

What comes next: Week 2, Topic 6 (HTTP + REST, controllers)

The first real API: routes, HTTP verbs, status codes, **DTOs** (which finally fix the 'entities leak' from topics 3 and 5), and controllers that call `IProjectService` with `async` and the request's `CancellationToken`. Checkpoint: GET/POST/PUT/DELETE work with sensible status codes.